CHAPTER 10: YEAR-1 CORE, PROFESSIONAL ETHICS & PRACTICE

“Make it work” is *not* the whole job

Software touches people's money, health and privacy. An engineer owes a duty to the **public**, to the **truth**, and to the people who use what we build, and in the AI era, a **person** always stays accountable.

TODAY: THE LAST CHAPTER, THEN GET READY FOR THE DEMO

Today: Ch10, then get ready for the demo and oral exam

Ch10 Ethics & Practice
3 concept checks

→

Demo day
your slot, what to present

→

Scoring
demo, oral, contribution

→

Exam practice
4 MCQ on Ch10

GOAL

Understand the concepts

Ch10 is on the final exam (Ch1 to Ch10, English MCQ with 4 options + short answer).

CONCEPT CHECKS

Answer together

After each topic: a short situation, the whole room answers at once. Not graded.

YOUR TEAM

Find your slot

13 teams on Mon 9 Nov, 13 on Mon 16 Nov, 12 minutes each. Know your time and what to show.

This chapter ties straight into your **AI-Use Statement** and the **oral exam**. It also includes a case study on **AI backdoors**.

THE GOLDEN THREAD: THE FINAL CHAPTER

From how we build to who we build for

Problem Ch1 → FR / NFR Ch3 → Model Ch4 → Design Ch5 → Code Ch6 → Test Ch7 → Quality Ch8 → PM Ch9 → Ethics Ch10

IDEAS 1 AND 2

Ethics & data

The ACM/IEEE Code, four everyday issues, and data ethics: can vs should.

IDEAS 3 AND 4

Responsible AI & practice

A person stays accountable, with an AI backdoor case study; habits that make you easy to work with.

IDEA 5

Dev tools

IDE, Git and GitHub, tests, and a taste of CI.

Ch8 asked “can it be trusted?”, Ch9 asked “how does the team finish?”. Ch10 asks **“should we build it this way, and who answers for it?”**

IDEA 1: DUTIES BEYOND “IT COMPILES”

Professional codes spell out what we owe

PUBLIC INTEREST

People come first

Act consistently with public health, safety and welfare, above your own or your client's convenience.

HONESTY

Be truthful

Don't overstate what the software does, hide known defects, or take credit for others' work.

COMPETENCE & CARE

Do work you can stand behind

Only sign off on work you believe is right, and keep learning. The **ACM/IEEE Code** is the shared standard.

IN PLAIN TERMS

A profession is a promise. Doctors, engineers and now software builders hold power over people's lives. The code of ethics is how we keep that trust.

IDEA 1A: WHY SOFTWARE ENGINEERING NEEDED ITS OWN CODE

Two societies, one code for a new profession

1993
IEEE-CS steering committee; ACM commission

→

1994
Joint ACM + IEEE-CS committee

→

1997
Draft in *Comm. ACM*, for comments

→

1999
Version 5.2 approved

THE GOAL

Make SE a profession

The joint committee set out to define standard terms, a body of knowledge, curricula and **ethical standards** for software engineers.

WHAT A CODE DOES

Educate, inspire, inform

It tells members what peers expect, tells the **public** what the profession is responsible for, and gives practical advice. It is **not a law**.

WHO IT COVERS

Including students

Practitioners, educators, managers, policy makers, **trainees and students** of the profession. That means you, now.

> THE PREAMBLE, IN ONE LINE
> People who build software have real chances "to do good and to cause harm", and to enable others to do either. The code exists so that power is used for good.

Source: Gotterbarn, Miller & Rogerson, "Software Engineering Code of Ethics," *Comm. ACM* 40(11), Nov 1997 (draft for comment). Final version 5.2 approved 1999.

IDEA 1B: THE EIGHT PRINCIPLES (ACM/IEEE-CS, 1999)

The Software Engineering Code of Ethics

1
Public
Act in the public interest.

2
Client & employer
Serve them, within the public interest.

3
Product
Meet the highest standards you can.

4
Judgment
Keep it independent and honest.

5
Management
Lead ethically.

6
Profession
Protect its integrity and reputation.

7
Colleagues
Be fair and supportive; credit their work.

8
Self
Keep learning, keep ethical.

Source: ACM/IEEE-CS Software Engineering Code of Ethics (Gotterbarn, Miller & Rogerson, *Comm. ACM*, 1999); Sommerville 9e, Fig 1.3. The 1997 draft listed Product first; the approved 1999 version moved **Public** to the top.

> IN PLAIN TERMS
> Read the **first word** of each line: Public comes first, then the people you serve, the product, your own judgment. Ethics radiates outward, starting from "do no harm to the public."

IDEA 1C: HOW THE CODE IS BUILT

Every principle works at three levels

LEVEL 1: ASPIRE

To be human

Values every person shares, such as honesty and fairness. They set a direction and need your own ethical judgment.

LEVEL 2: EXPECT

To be professional

Duties every professional owes, because people **affected by your work** deserve special care.

LEVEL 3: DEMAND

To use good practices

Specific, often measurable behaviours of software engineering today, close to the current state of the art.

LEVEL	EXAMPLE FOR “PUBLIC” (OUR ILLUSTRATION OF THE IDEA)
Aspire	Be fair and truthful in everything you say about the software.
Expect	Accept full responsibility for your own work, including the parts AI drafted.
Demand	Approve software only if you have good grounds to believe it is safe, meets its specification and has passed appropriate tests.

Source: Gotterbarn, Miller & Rogerson (1997), preamble. The code does not label each clause with a level; the table shows how the idea applies.

IDEA 1D: WHAT EACH PRINCIPLE ASKS (1 TO 4)

Principles 1 to 4: what they ask you to do

PRINCIPLE	KEY CLAUSES (PARAPHRASED)	IN THIS COURSE
1 Public	Disclose any danger the software may cause; approve only software you believe is safe and tested; never put your own, your employer's or a client's interest ahead of the public's; accept full responsibility for your work.	"The AI wrote it" is not a defence; you own what you submit.
2 Client & employer	Work only within your competence; keep their information confidential (unless the public is at risk); tell them early if a project is likely to fail, overrun, or break copyright; never use illegally obtained software.	Tell the team *before* the deadline; no pirated tools or unlicensed code.
3 Product	Specifications documented and approved; realistic estimates with a risk assessment; adequate testing, debugging and review; respect the privacy of users; use only accurate data from legal sources.	SRS + acceptance criteria (Ch3), tests (Ch7), PDPA (Ch8), risk (Ch9).
4 Judgment	Stay objective when evaluating software; sign only documents within your competence; reject bribery; disclose conflicts of interest; temper technical judgment with human values.	Never sign a test report you did not run.

Clauses paraphrased from Gotterbarn, Miller & Rogerson (1997). Principle numbers follow the approved 1999 version (Sommerville Fig 1.3).

IDEA 1E: WHAT EACH PRINCIPLE ASKS (5 TO 8)

Principles 5 to 8: what they ask you to do

PRINCIPLE	KEY CLAUSES (PARAPHRASED)	IN THIS COURSE
5 Management	Tell people the standards before holding them to them; assign work by education and experience, while helping people grow; agree fairly who owns what; never ask anyone to break the code.	Team leads rotate (Ch9); agree roles and ownership in the charter.
6 Profession	Be accurate about what the software does, avoiding deceptive or misleading claims; take responsibility for finding, fixing and reporting errors; obey the law, as far as it is consistent with public welfare.	At the demo, show what works; don't oversell.
7 Colleagues	Credit fully the work of others; review others' work objectively, candidly and with a record; give colleagues a fair hearing; ask experts when outside your competence.	Pull-request reviews (Ch6); honest credit in the AI-Use Statement.
8 Self	Keep learning design, development, testing and management; improve your documentation; know the law and the code that govern your work.	AI makes learning faster, not optional.

Clauses paraphrased from Gotterbarn, Miller & Rogerson (1997). Principle numbers follow the approved 1999 version (Sommerville Fig 1.3).

IDEA 1F: USING THE CODE WHEN IT GETS HARD

The code is not an algorithm: five questions to ask

1. **Who is affected?**
   Think broadly: users, third parties, people who never touch the app.
2. **Are we treating people with respect?**
   You and your colleagues, towards every person involved.
3. **How would a well-informed public see it?**
   If they knew all the facts, would they accept your decision?
4. **What happens to the least empowered?**
   The people with the least power to complain or protect themselves.
5. **Would an ideal professional do this?**
   Would respected, experienced peers who knew the facts judge it worthy?

TRY IT

Ship Friday, or fix the leak?

Affected: every user whose data can leak. Least empowered: users who never chose this app. Informed public: would not accept “we knew, but the deadline came first.” **Answer: report the risk and delay or fix (Principle 1).**

WHY FIVE QUESTIONS, NOT A RULEBOOK

Principles can conflict with each other or with other rules. The code gives no automatic answer; it gives you a **way to judge**, and a written standard the whole team can point to when they say no.

Source: Gotterbarn, Miller & Rogerson (1997), preamble. Not exhaustive: the code warns against using any single clause in isolation to justify a mistake.

CONCEPT CHECK 1: THE EIGHT PRINCIPLES

Which principle does it break?

SITUATION	PRINCIPLE	WHY
Your manager says ship on Friday, although the team knows a bug can leak users' data.	**1 Public**	People's data before the deadline (also 3 and 4)
You sign a test report saying all tests passed, but you never ran them.	**4 Judgment**	Sign only what you believe is true
You present a classmate's module as your own work in the demo.	**7 Colleagues**	Credit the work of others
A team lead tells juniors to hide known defects from the client.	**5 Management**	Leaders must lead ethically

FOLLOW-UP

Doesn't Principle 2 say “serve your employer”? **Yes, but only within the public interest. Principle 1 comes first.**

IDEA 1G: WHERE ETHICS SHOWS UP (SOMMERVILLE §1.2)

Four issues you'll meet in ordinary work

#1

Confidentiality

Respect your employer's and client's information, with or without a signed agreement.

#2

Competence

Don't misrepresent your skill, and don't take on work beyond it.

#3

IP rights

Know and respect patents, copyright and licences. Protect what isn't yours.

#4

Computer misuse

Don't use your skills to harm others' systems, from time-wasting to malware.

CONNECTS TO THIS COURSE

Confidentiality + IP = why you never commit secrets or copy code without a licence. Competence + misuse = why “the AI wrote it” never excuses harm.

Source: Sommerville 9e, §1.2 Software engineering ethics.

IDEA 2: CAN VS SHOULD

Just because you *can* collect it doesn't mean you *should*

MINIMISE

Collect the least

Only what the feature truly needs. Every extra field is a liability you now have to protect.

PROTECT

Guard what you hold

Secure it, limit who can see it, and honour people's right to see, correct and delete (PDPA, Ch8).

BE TRANSPARENT

Say what and why

Tell people what you collect and how it's used, in plain language, before they hand it over.

CARRIES CH8 FORWARD

Security was the *how*; data ethics is the *whether*. The most private data is the data you chose never to collect. Which fields in your prototype are not really needed?

CONCEPT CHECK 2: EVERYDAY ISSUES & DATA ETHICS

Which issue or data rule is it?

SITUATION	ISSUE OR RULE
You post a screenshot of a client's internal dashboard in a public Discord to ask for help.	**Confidentiality**
You paste code from a GitHub repo that has no licence into your product.	**IP rights**
After one tutorial, you tell a client you are an expert and take the job.	**Competence**
A study-reminder app asks for your national ID number and birthday at sign-up.	**Minimise**
The app shares usage data with an ad partner, but the privacy notice never says so.	**Be transparent**

FOLLOW-UP

The last two are legal and technical problems too. **Ch8 PDPA said what the law requires; Ch10 asks whether we should collect it at all.**

IDEA 3: AI-FIRST, BUT A PERSON IS ACCOUNTABLE

AI can be wrong or biased, and it cannot be responsible

DO

Keep a human sign-off
A person reads, tests and approves. Accountability can't be delegated to a model.

Be transparent
Disclose where AI was used. That is exactly what your **AI-Use Statement** is for.

Watch for bias and error
AI reflects its training data. Check outputs against reality, especially about people.

DON'T

"The AI said so"
Not a defence. If you shipped it, you own it, bug and bias included.

Paste private data into prompts
Treat a prompt as public. Never feed real personal data or secrets.

Pass AI work off as fully your own
Honesty (Idea 1) applies to how the work was made, too.

IN PLAIN TERMS

AI is like a **very fast intern**: brilliant first drafts, but you sign the work before it goes out. If it's wrong, the boss (you) answers for it, never the intern. This is one of the four oral-exam criteria.

CONCEPT CHECK 3: RESPONSIBLE AI

Responsible AI use, or not?

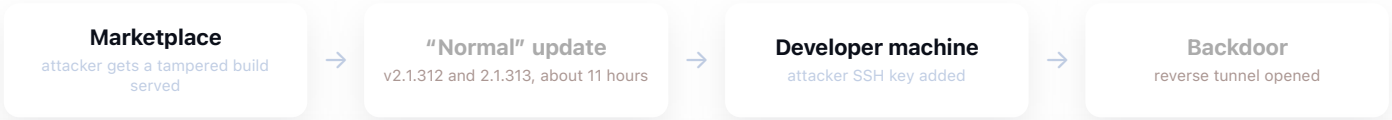
SITUATION	VERDICT	WHY
AI drafts your unit tests; you run them and fix two wrong expected values.	**Responsible**	A person checked and signed off
You paste a real clinic's patient list into a chatbot to “make test data”.	**Not OK**	Real personal data in a prompt
The AI-Use Statement says “no AI used”, but AI generated the UI layout.	**Not OK**	Honesty: disclose, don't hide
AI-written code leaks data; the team's report says “the AI caused it”.	**Not OK**	Accountability stays with the team

FOLLOW-UP

Is using AI itself the problem? **No. Using AI well is expected in this course. Hiding it, or shipping what you can't explain, is the problem.**

CASE STUDY: IN THE NEWS, OCTOBER 2026

A trusted AI tool update carried a backdoor



THE INCIDENT, 29 OCT 2026

Claude Code extension on the Cursor Marketplace

Cursor says at most 5,000 users received it and other extensions were not affected. Cursor says the tampered version was not developed by Anthropic; Anthropic says none of its systems were compromised. How the attacker got in has not been stated in the sources we checked.

SAME MONTH: TWO RELATED CASES

Packages and model weights

LiteLLM 1.107.0 (21 Oct): stolen GitHub tokens ran the release workflow; the PyPI build had a credential stealer and backdoor. **Imperial College study:** a tampered speech model reacted to one chosen voice. Research in a sandbox, not a real attack.

BACKDOOR, IN PLAIN TERMS

A hidden way in that bypasses normal login. The tool looks normal and has your permissions, so **trust is the attack path**.

Sources: Cursor security notice; Microsoft Security Blog, 30 Oct 2026; SafeDep; Help Net Security, 22 Oct 2026. Developing story: check for updates.

CASE STUDY: READ THE ORIGINAL REPORTS

In the news, October 2026

CURSOR, OFFICIAL NOTICE

Security notice: malicious version of Anthropic.claude-code

Primary source. See also Microsoft's analysis.

Cursor notice | Microsoft

SAFEDEP

October 2026 Security Incident Review

Covers the Cursor Marketplace and LiteLLM incidents.

Read the review

HELP NET SECURITY, 22 OCT 2026

Say a few words, and a rigged AI asks for $5,000

Imperial College research, tested in a sandbox.

Read the report

READ LIKE AN ENGINEER

Who is the source? What is confirmed, and what is still unknown? Is it a real incident or a research test?

CASE STUDY: DISCUSS, THEN USE THE CODE

Who is responsible, and what would you do?

QUESTION	CH10 LINK	A PROFESSIONAL RESPONSE
Who is affected?	**Five questions**	Developers, and everyone whose data or code sits on their machines.
You installed the update. What do you do first?	**1 Public, 6 Profession**	Report it, follow the vendor's notice, check SSH keys and tunnels, rotate credentials. Uninstalling alone may not be enough.
"The tool was official." A full defence?	**Accountability**	No. It explains how it happened, but your team still owns what it ships.
What should a team change?	**3 Product, Ch8**	Check where tools come from, delay brand-new versions, use short-lived scoped secrets, limit what AI agents can do.

> KEEP USING AI, BUT
> Treat AI tools, packages and models as **supply-chain dependencies**: useful, powerful, and never trusted without checks. A human stays accountable.

CONCEPT: BACKDOOR IN CODE

A backdoor can also arrive in code AI writes for you

EXAMPLE: LOGIN FUNCTION

`if password == "dev-2569": return True`

Nobody asked for this line, yet all login tests still pass. Anyone who knows the value gets in without a real password.

WHY TESTS MISS IT

Tests check what should work

They rarely check for an extra way in. That is why you **read the code** before committing, and why review still matters.

WHAT TO DO

Remove the line, tell the team, and test the login again. Bug or backdoor, intended or not, the risk is the same, and the team that ships it owns it.

CASE STUDY: WHAT ABOUT A LOCAL MODEL?

“Local” does not mean “trusted”

ROUTE	HOW IT WORKS	CAN IT SEND DATA OUT?
Hidden code in the file	Pickle files or `trust_remote_code` can run code when loaded.	Yes, like ordinary malware. No trigger needed.
Backdoor in the weights	Training makes the model change its output on a trigger.	Not alone. Weights are numbers; they need software around them.
Model + tools or UI	A triggered output calls a tool, or puts data in a link or image URL.	Yes, if the tool has permission or the UI loads the URL.

REDUCE THE RISK

Check the publisher and hash; prefer `safetensors`; avoid `trust_remote_code`; use a sandbox with no network; give tools the least privilege; a person confirms risky actions.

THE LIMIT

A file scan finds hidden code, not behaviour learned in training. Design so that **even a triggered model cannot get data out**.

IDEA 4: HABITS THAT MATTER AS MUCH AS CODE

Being easy to work with is an engineering skill

COMMUNICATE

Clearly and early

Say what you're doing, and say when you're stuck, before the deadline.

COMMIT

Keep your word

Deliver what you said, or renegotiate early. Reliability is reputation.

FEEDBACK

Give and take it

Critique the work, not the person; receive it without defensiveness.

DOCUMENT

Leave a trail

A README and clear commits so others, and future-you, can follow.

WHY IT'S GRADED

These are the habits the oral exam and the GitHub contribution factor reward: the parts of engineering that no model does for you.

IDEA 5: WHAT A REAL TEAM BUILDS WITH

The toolkit, and where CI fits



You already use the IDE, Git and tests. **CI (Continuous Integration)** is the new idea: a service that **runs your tests on every push**, so a broken change is caught in minutes, by a machine, before it reaches the team.

> IN PLAIN TERMS
> CI is a **tireless teammate** who re-runs every test the moment you push. Our prototypes are static, so you don't need to set up CI; **GitHub Pages**, which republishes your site on every push, is the same idea at the Deploy step.

DO THIS WITH YOUR TEAM

Finish like a professional

PROJECT STEP

Keep the AI-Use Statement honest

A milestone without it is capped at 50%, and it is your transparency on record.

Collect the minimum

Trim the personal data your prototype shows or asks for before the demo; be ready to justify every field. Use made-up data only.

AI-FIRST: OWN IT

Be ready to explain any line

At the oral exam, “the AI wrote it” is not an answer. Understanding is.

Disclose, don't hide

Using AI well is expected and rewarded; hiding it is the problem.

REFERENCES: WHERE THIS COMES FROM

Textbook & source references

1 **Sommerville, I. (2011) Software Engineering, 9e**
§1.2 Software engineering ethics: confidentiality, competence, IP, misuse; Fig 1.3 the ACM/IEEE Code.

2 **ACM/IEEE-CS Code of Ethics (1999)**
Gotterbarn, Miller & Rogerson: draft in *Comm. ACM* 40(11), 1997; version 5.2 approved 1999. Eight principles, three levels of obligation.

3 **Leveson & Turner (1993)**
An investigation of the Therac-25 accidents: accountability and safety, *IEEE Computer* 26(7).

4 **US EPA (2015)**
Volkswagen defeat-device notice: dishonest code written to cheat an emissions test.

5 **PDPA B.E. 2562 (2019)**
The data-protection duty behind “collect the minimum, be transparent.”

Before the final exam: read the eight principles and Sommerville §1.2. Your AI-Use Statement is Principle 4 (honest judgment) in practice.

CHAPTER 10: IN FOUR LINES

Build for people, stay accountable, leave a clear trail

ETHICS

Duties beyond the code

Public interest, honesty, competence. The ACM/IEEE Code is the shared standard; Public comes first.

DATA

Can vs should

Minimise, protect, be transparent. The safest data is what you never collected.

AI

A person is accountable

Disclose use, keep a human sign-off, watch for bias. Check AI tools and models like any dependency. “The AI said so” is never a defence.

PRACTICE

Habits and tools

Communicate, commit, give feedback, document; let IDE, Git, tests and CI carry the work.

DEMO + ORAL EXAM: WHEN IS YOUR TEAM?

13 teams a day, 12 minutes each, in our usual class slot

Group A: Mon 9 Nov, 12:00 to 14:50

SLOT	GROUP	TEAM
12:05 to 12:17	**Group 1**	Aura
12:17 to 12:29	**Group 2**	5 idoiots
12:29 to 12:41	**Group 3**	I'm so hungry ahh
12:41 to 12:53	**Group 4**	Apollo
12:53 to 13:05	**Group 5**	Viounique
13:05 to 13:17	**Group 6**	dek70
13:17 to 13:29	**Group 7**	Gamza007
13:29 to 13:41	**Group 8**	MedMind
13:41 to 13:53	**Group 9**	Hew Khao Mak
13:53 to 14:05	**Group 10**	Strategic
14:05 to 14:17	**Group 11**	ILY3000
14:17 to 14:29	**Group 12**	PhomPen Duke
14:29 to 14:41	**Group 13**	Socrates and Skeletons

Group B: Mon 16 Nov, 12:00 to 14:50

SLOT	GROUP	TEAM
12:05 to 12:17	**Group 14**	Larb Lamphon
12:17 to 12:29	**Group 15**	I don't know
12:29 to 12:41	**Group 16**	16RideRight
12:41 to 12:53	**Group 17**	ByteSquad
12:53 to 13:05	**Group 18**	MFU888
13:05 to 13:17	**Group 19**	Yhar Bid
13:17 to 13:29	**Group 20**	Home sick
13:29 to 13:41	**Group 21**	lumen Inc.
13:41 to 13:53	**Group 22**	Hallo world
13:53 to 14:05	**Group 23**	Trauma squad
14:05 to 14:17	**Group 24**	team 24
14:17 to 14:29	**Group 25**	Team25
14:29 to 14:41	**Group 26**	The last one

12:00 to 12:05 set-up and briefing; 14:41 to 14:50 spare time. Arrive 10 minutes before your slot with the prototype already open. The whole team must be present: the oral part is individual.

YOUR 12 MINUTES: 5 MIN DEMO + 6 MIN QUESTIONS (+1 MIN SET-UP)

Present in this order: follow the Golden Thread

#	TOPIC (5-MIN DEMO)	TIME
1	**Problem and user**: who has the pain, and what it is (M1)	0:30
2	**Must FRs**: the MVP list from your SRS (M2)	0:30
3	**Live click-through**: every Must FR happy path + 1 unhappy path (empty or error). Point: "this screen = FR-3 = this diagram = this acceptance test"	3:00
4	**AI use**: where AI helped, what you checked and changed	0:30
5	**Next step**: one thing you would improve, and why	0:30

Q

6 min questions: every member answers

1 to 2 questions each, about **your own part** and your AI use. Scored on the oral criteria (3 slides on).

HAVE READY BEFORE YOUR SLOT

Prototype opens in a browser from the repo (no server, no DB). Demo slides, Final Report with the Golden Thread table, and the AI-Use Statement in the repo. Laptop and display adapter tested.

OVER TIME

At 5:00 the demo stops and questions start. Unshown features count as not shown.

TEAM SCORE: M3 FINAL PROTOTYPE + DEMO = 30 POINTS (15% OF THE COURSE)

Marks are for the Golden Thread, not for a pretty app

CRITERION	POINTS	FULL MARKS LOOK LIKE
Clickable prototype covers every Must FR	**12**	Every Must-FR happy path is clickable end to end (no real DB), plus an unhappy path (empty or error state)
Golden Thread coherence	**6**	You can point: this feature = FR-3 = this diagram = this acceptance test
Demo and presentation	**6**	Tells the Problem to Solution story, on time, answers "why" questions
Engineering and AI-first process	**6**	Clean repo (prototype + docs inside), README says how to open it, honest AI-Use Statement

AI-USE GATE
No AI-Use Statement: M3 capped at 50%.

COHERENCE GATE
Problem, design and prototype about different things: capped at 50%.

LATE OR MISSING
Late = half. Not handed in = 0. Heavy gaps = returned for revision.

FINAL PROJECT MARKING RUBRIC: M3 = 30 POINTS

What each score level looks like

CRITERION	STRONG	DEVELOPING	WEAK	0
Prototype, Must FRs (12)	**10 to 12:** every Must FR clickable, plus an unhappy path	**6 to 9:** most Must FRs, or no unhappy path	**1 to 5:** few flows work end to end	Won't open
Golden Thread (6)	**5 to 6:** every Must FR maps to diagram and test	**3 to 4:** some links missing	**1 to 2:** mostly disconnected	No table
Demo (6)	**5 to 6:** clear story, on time, answers "why"	**3 to 4:** clear, but weak on "why"	**1 to 2:** unclear or incomplete	No demo
Engineering and AI (6)	**5 to 6:** clean repo, working README, specific AI-Use Statement	**3 to 4:** complete, but thin	**1 to 2:** items missing or unclear	Nothing usable

Then apply: no AI-Use Statement or broken thread = capped at 50%; late = half; contribution factor per person.

INDIVIDUAL SCORE: ORAL EXAM = 4 CRITERIA X 0 TO 5 = 20, SCALED TO 10%

Explain your own part, and how you used AI

CRITERION	5 STRONG	3 OK	1 WEAK	0
Understands the concept	Explains clearly, correct	Mostly correct	Confused	No idea
Can explain own work	Owns it, in detail	Partial	Shaky	Can't
AI use: transparent and critical	What AI did, what you changed, and why	Some awareness	"AI did it"	Hides or denies
Communication	Clear, confident	Adequate	Unclear	None

RED FLAG

Can't explain work your team credits to you: "can explain own work" = 0, and it is noted as possible undisclosed AI use or free-riding.

SAMPLE QUESTIONS

Pick one feature: which problem, which FR, which test proves it? Walk me through your part. Where did you use AI, and what did you change? What would break if this input changed?

YOUR PROJECT MARK = TEAM SCORE X CONTRIBUTION FACTOR (GITHUB ONLY, NO PEER REVIEW)

Start at 1.00; it only goes down, with evidence

FACTOR	LEVEL	MEANING
1.00	Normal (most students)	Visible work in the repo matching your role in the Team Charter
0.70	Partial	Thin or last-minute GitHub work, after a chance to explain
0.40	Barely any	No verifiable work in the repo across the project

A factor is lowered only when both are true

1. **GitHub shows little or no work**
   Commits, PRs, issues, committed docs, diagrams, prototype. Judged on meaning, not commit count.

2. **You saw the evidence and had 3 days to explain**
   Non-code work (design, docs, PM, testing) must be pointable in the repo.

DO IT NOW

Open your repo: Insights, Contributors. If your work is not there under your own account, it does not count. The instructor marks the demo, the oral exam and the factor on one score sheet per team.

EXAM REVIEW: CH10 ON ONE PAGE

What the final exam can ask from Ch10

ETHICS & DATA

Code	ACM/IEEE-CS, 8 principles: Public, Client & employer, Product, Judgment, Management, Profession, Colleagues, Self. Public comes first; Principle 2 is within the public interest. Three levels: aspire, expect, demand. Not an algorithm: ask who is affected, and how the least empowered fare.
Everyday	Sommerville §1.2: confidentiality, competence, IP rights, computer misuse.
Data	Can vs should: minimise, protect, be transparent.

AI & PRACTICE

AI	Human sign-off, disclose use, watch for bias; never paste private data; “the AI said so” is not a defence.
Habits	Communicate, commit, feedback, document.
Tools	IDE, Git/GitHub, tests; CI runs the tests automatically on every push; deploy.

Final exam: Ch1 to Ch10, English MCQ (4 options) + short answer. You will be asked to explain, choose or classify.

EXAM PRACTICE: CH10 MCQ 1 TO 4 (ALONE, ON YOUR OWN PAPER, NOT GRADED)

Practice like the real exam

1. **In the ACM/IEEE-CS Software Engineering Code of Ethics, which principle comes first?**

A. Client and employer.
B. Product.
C. Public.
D. Self.

2. **A study-reminder prototype asks every user for their national ID number. Which data-ethics principle does this break most directly?**

A. Transparency.
B. Data minimisation.
C. Competence.
D. Intellectual property.

3. **A bug in AI-generated code exposes users' data after release. Who is accountable?**

A. The company that trained the AI model.
B. No one, because the code was generated automatically.
C. The engineers who reviewed and shipped the code.
D. The user who reported the bug.

4. **What does Continuous Integration (CI) do?**

A. It writes the unit tests for you.
B. It runs the tests automatically on every push.
C. It deploys the system once, at the end of the project.
D. It replaces code review.

About 4 minutes. Answers are revealed one by one afterwards.

THAT'S CH10, AND THE LAST CONTENT CLASS

Next: demo, oral exam, final exam

CRITIQUE PAPERS

8 sheets this term

No Critique today. You started by spotting AI mistakes and finished by asking your own questions.

TEAM DELIVERABLES

Final Report + AI-Use Statement

Due date to be confirmed. Golden Thread table, demo slides, repo frozen. Commit from your own account.

DEMO + ORAL EXAM

Group A Mon 9 Nov

Group B Mon 16 Nov, 12:00 to 14:50, 12 minutes per team. Re-read your team's AI-Use Statement first.

FINAL EXAM

23 Nov to 4 Dec

Ch1 to Ch10, English MCQ + short answer. The Ch10 self-study handbook is posted in Classroom.